

1. Explain Technical Debt. What Are the Major Causes of Technical Debt, and How Can Organizations Minimize It During Software Evolution?

Introduction

Technical debt refers to the future cost and additional effort created when a software team chooses a quick or convenient solution instead of a better long-term solution.

The concept is similar to financial debt. A team may gain a short-term benefit by implementing a quick solution, but if the issue is not corrected later, the cost of maintaining and modifying the software increases.

For example, a developer may implement a simple solution to meet a deadline instead of designing a scalable solution. Initially, development is faster, but future developers may need additional time to understand, modify, and maintain the code.

1.1 Example of Technical Debt

Suppose a company needs to add a search feature quickly.

Instead of designing an efficient search mechanism, the developer writes a simple linear search: for every product:

compare product with search text

This may work when there are 1,000 products.

However, when the system grows to 10 million products, the search becomes slow.

The organization now needs to redesign the search mechanism, optimize the database, and modify the application.

The additional effort represents **technical debt**.

1.2 Major Causes of Technical Debt

1. Tight Project Deadlines

When teams have strict deadlines, they may choose quick solutions instead of properly designing the system.

Example:

A developer skips refactoring because a new feature must be delivered immediately.

2. Lack of Proper Planning

Poor architecture and inadequate requirements analysis can lead to technical debt.

If future scalability is not considered during design, the system may become difficult to extend.

3. Poorly Written Code

Code that is difficult to understand, duplicated, or unnecessarily complex increases maintenance effort.

Examples include:

- Very long functions
- Poor naming
- Duplicate code
- Unnecessary complexity

- Lack of modularity

4. Inadequate Testing

If testing is insufficient, defects remain in the system.

Developers may later spend significant time fixing old defects while implementing new features.

5. Lack of Documentation

When architecture, APIs, and important design decisions are not documented, new developers need more time to understand the system.

This increases maintenance costs.

6. Outdated Technologies and Dependencies

Software may depend on old frameworks, libraries, or programming languages.

Over time, outdated dependencies can create:

- Security vulnerabilities
- Compatibility problems
- Performance issues
- Difficulties finding developer support

7. Frequent Requirement Changes

Software requirements can change continuously.

If changes are implemented without refactoring the architecture, the codebase can become increasingly complex.

8. Lack of Code Reviews

When code is merged without proper review, poor design decisions and coding problems may remain unnoticed.

9. Copy-and-Paste Development

Developers may copy existing code instead of creating reusable components.

This creates duplicate logic that must be maintained in multiple places.

1.3 Types of Technical Debt

Technical debt can occur in several forms.

Design Debt

Poor architectural decisions make future changes difficult.

Code Debt

Poor-quality or complex source code increases maintenance effort.

Testing Debt

Insufficient automated tests make future changes risky.

Documentation Debt

Missing or outdated documentation makes the system difficult to understand.

Dependency Debt

Old libraries and frameworks create maintenance and security problems.

Infrastructure Debt

Outdated deployment and infrastructure configurations can increase operational costs.

1.4 Effects of Technical Debt

Technical debt can cause:

1. Increased maintenance costs.
2. Slower development.
3. More software defects.
4. Security vulnerabilities.
5. Difficult system upgrades.
6. Reduced developer productivity.
7. Poor application performance.
8. Increased risk during new feature development.
9. Difficulty onboarding new developers.
10. Reduced customer satisfaction.

1.5 How Organizations Can Minimize Technical Debt

1. Perform Code Reviews

Every important code change should be reviewed by other developers.

Code reviews help identify:

- Poor design
- Security issues
- Duplicate code
- Maintainability problems

2. Use Automated Testing

Organizations should maintain automated:

- Unit tests
- Integration tests
- API tests
- Regression tests

Automated testing allows developers to safely modify existing software.

3. Refactor Regularly

Refactoring means improving the internal structure of software without changing its external behavior.

Examples:

Before:

Large Function

↓

1000 lines of code

After:

Large Function

↓

Multiple Smaller Functions



Reusable Components

Regular refactoring prevents small problems from becoming major technical debt.

4. Maintain Documentation

Architecture diagrams, API documentation, setup instructions, and important design decisions should be kept updated.

5. Update Dependencies

Organizations should regularly update frameworks, libraries, operating systems, and security components.

Dependency scanning tools can also identify vulnerable packages.

6. Allocate Time for Technical Debt

Technical debt should be treated as part of normal software maintenance.

For example, a development team can reserve a percentage of every sprint for:

- Refactoring
- Bug fixing
- Dependency updates
- Documentation
- Performance improvements

7. Use CI/CD

Continuous Integration and Continuous Delivery automatically test and validate code changes.

A typical process is:

Code Change



Build



Unit Tests



Integration Tests



Security Checks



Code Review



Deployment

This helps detect problems early.

8. Use Good Architecture

Organizations should select architectures that support expected future requirements.

For example, modular architecture allows individual components to be changed without rewriting the entire system.

9. Track Technical Debt

Technical debt items should be recorded in project management tools.

Each debt item should include:

- Description
- Impact
- Priority
- Estimated effort
- Responsible team
- Planned resolution date

1.6 Technical Debt Management During Software Evolution

Software continuously evolves through new features, bug fixes, performance improvements, and security updates.

A good evolution strategy is:

New Requirement



Impact Analysis



Implementation



Automated Testing



Code Review



Refactoring if Required



Deployment



Monitoring

Organizations should continuously identify and remove technical debt instead of allowing it to accumulate.

Conclusion

Technical debt is not necessarily bad because sometimes a temporary compromise is necessary to meet business requirements or deadlines. However, unmanaged technical debt can significantly increase future development and maintenance costs.

Organizations can minimize technical debt through proper planning, code reviews, automated testing, continuous refactoring, documentation, dependency management, CI/CD, and regular monitoring. Managing technical debt throughout software evolution helps maintain software quality, scalability, security, and developer productivity.

2. Explain How AI/ML and Edge Computing Are Transforming Modern Software Engineering. Provide One Practical Example for Each.

Introduction

Modern software engineering is changing rapidly because of technologies such as **Artificial Intelligence/Machine Learning (AI/ML)** and **Edge Computing**.

AI/ML allows software systems to learn from data, automate tasks, predict outcomes, and assist developers.

Edge Computing moves computation closer to the devices or users generating the data. This reduces network latency and decreases the need to send all data to a centralized cloud server.

2.1 AI/ML in Modern Software Engineering

Artificial Intelligence enables software systems to perform tasks that traditionally required human decision-making.

Machine Learning allows systems to learn patterns from data and make predictions or decisions.

AI/ML is transforming software engineering in several areas.

2.2 AI-Assisted Software Development

AI tools can assist developers with:

- Code generation
- Code completion
- Bug detection
- Code explanation
- Test-case generation
- Documentation
- Refactoring suggestions

For example, a developer can describe a required function in natural language and an AI system can generate an initial implementation.

The developer can then review, test, and modify the generated code.

2.3 AI-Based Testing

AI/ML can help generate and prioritize test cases.

For example, machine learning can analyze previous software failures and identify areas of the application that are more likely to contain defects.

This allows testing resources to focus on high-risk areas.

2.4 Predictive Maintenance

AI can analyze application logs and performance metrics to identify patterns that may indicate future failures.

Example:

CPU Usage ↑

Memory Usage ↑

Error Rate ↑

↓

AI Model

↓

Possible Failure Predicted



Engineering Team Takes Action

This can reduce downtime.

2.5 Intelligent Security

AI/ML can identify unusual behavior and potential security threats.

For example, a machine-learning system can detect an unusual number of login attempts from an account and flag the activity for investigation.

2.6 Practical Example of AI/ML

Example: AI-Based Software Defect Prediction

A large software company has thousands of source-code files and historical bug reports.

An ML model is trained using information such as:

- Number of code changes
- Code complexity
- Previous defects
- Number of developers modifying a file
- Code size

The model predicts which components are more likely to contain defects.

Example:

Component A → Low Risk

Component B → Medium Risk

Component C → High Risk

Developers can prioritize code review and testing for Component C.

Benefits

- Faster testing
- Better resource allocation
- Early defect detection
- Reduced maintenance cost
- Improved software quality

2.7 Edge Computing

Edge Computing is a computing model in which data processing takes place close to the source of data rather than sending all information to a centralized cloud server.

Traditional cloud model:

Device



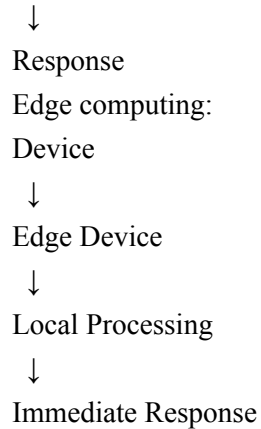
Internet



Cloud Server



Processing



2.8 Advantages of Edge Computing

1. Low Latency

Data can be processed near the user or device.

This is important for applications that require immediate responses.

2. Reduced Bandwidth Usage

Instead of sending all raw data to the cloud, only important information may be transmitted.

3. Improved Reliability

Some processing can continue even when the connection to the cloud is temporarily unavailable.

4. Better Privacy

Sensitive data can sometimes be processed locally instead of sending it entirely to a remote server.

5. Real-Time Processing

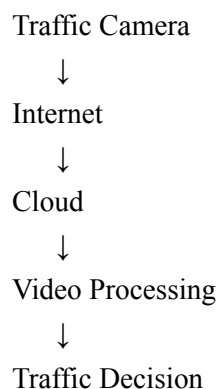
Edge computing is useful for applications where decisions must be made within milliseconds.

2.9 Practical Example of Edge Computing

Example: Smart Traffic Management

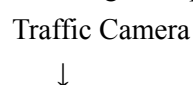
Consider a smart traffic system using cameras at road intersections.

A traditional system might send all camera video to a cloud server:



This can consume significant bandwidth and introduce latency.

With edge computing:



Edge Computer



Vehicle Detection



Traffic Analysis



Traffic Signal Control

The edge device can detect:

- Number of vehicles
- Traffic density
- Emergency vehicles
- Traffic congestion

The traffic signal can then be adjusted immediately.

Only summary information may be sent to the cloud for long-term analysis.

2.10 AI/ML and Edge Computing Together

AI/ML and Edge Computing can also be combined.

For example:

Camera



Edge Device



AI Model



Object Detection



Immediate Decision



Cloud



Long-Term Analytics

This approach is useful for:

- Autonomous vehicles
- Smart cities
- Industrial automation
- Healthcare monitoring
- Security systems
- Smart agriculture

2.11 AI/ML vs Edge Computing

Aspect	AI/ML	Edge Computing
Main purpose	Intelligent prediction and automation	Local/near-source processing
Major benefit	Automated decision-making	Low latency
Common use	Prediction and classification	Real-time processing
Example	Defect prediction	Smart traffic control
Main requirement	Data and ML models	Edge devices and connectivity

Conclusion

AI/ML is transforming software engineering by assisting development, testing, prediction, maintenance, and security. Edge Computing is transforming system architecture by moving processing closer to users and devices, reducing latency and bandwidth requirements.

AI/ML can make software more intelligent, while Edge Computing can make software more responsive and efficient. Together, they enable modern applications such as autonomous systems, smart cities, industrial automation, and real-time monitoring.

3. Case Study – Sustainable DevOps Practices for a Cloud-Based Microservices Application

Introduction

A company operates dozens of microservices in the cloud. As the number of services and users increases, the organization notices increasing resource utilization, cloud costs, and energy consumption.

Traditional DevOps focuses mainly on speed, reliability, and automation. **Sustainable DevOps**, also called Green DevOps, additionally considers energy efficiency and environmental impact.

Three important practices that can reduce both energy consumption and cloud cost are:

1. Resource optimization and autoscaling.
2. Efficient containerization and workload scheduling.
3. Monitoring and continuous optimization.

3.1 Practice 1 – Resource Optimization and Autoscaling

Cloud resources should match actual application demand.

A common problem is running too many virtual machines, containers, or Pods even when traffic is low.

For example:

Low Traffic

↓

20 Servers Running

↓

Low Utilization

↓

Wasted Resources

↓

Higher Cost + Energy Consumption

Autoscaling can dynamically adjust resources:

Low Traffic



Reduce Instances



Lower Resource Usage

High Traffic



Increase Instances



Maintain Performance

For Kubernetes-based microservices, a Horizontal Pod Autoscaler can increase or decrease the number of Pods based on CPU utilization or other metrics.

Example

Suppose a service normally requires:

10 Pods during peak hours

3 Pods during low traffic

Instead of running 10 Pods continuously, autoscaling can maintain approximately 3 Pods during low demand and increase the number when traffic rises.

Benefits

- Lower cloud costs.
- Reduced CPU and memory consumption.
- Lower energy usage.
- Better resource utilization.
- Automatic adaptation to workload changes.

3.2 Practice 2 – Efficient Containerization and Workload Scheduling

Containers allow applications to share host resources efficiently.

Instead of running a separate virtual machine for every microservice, multiple lightweight containers can run on appropriately sized infrastructure.

Example:

Traditional Approach

VM 1 → Service A

VM 2 → Service B

VM 3 → Service C

VM 4 → Service D

This may result in significant unused resources.

With efficient container orchestration:

Cloud Node

```
-----  
| Container A   |  
| Container B   |  
| Container C   |  
| Container D   |  
-----
```

The available resources can be utilized more efficiently.

Additional Techniques

Organizations can:

- Use lightweight container images.
- Remove unnecessary dependencies.
- Set CPU and memory requests and limits.
- Consolidate workloads when appropriate.
- Use efficient instance types.
- Schedule non-urgent workloads during periods of lower demand.

Benefits

- Better server utilization.
- Lower infrastructure requirements.
- Reduced energy consumption.
- Reduced cloud infrastructure cost.

3.3 Practice 3 – Monitoring, Measurement, and Continuous Optimization

What cannot be measured effectively cannot be optimized.

Organizations should monitor:

- CPU utilization
- Memory utilization
- Network traffic
- Storage usage
- Container utilization
- Application response time
- Request rate
- Cloud resource usage
- Cost per service

A monitoring system can identify services that consume excessive resources.

Example:

Service A → CPU 25%

Service B → CPU 30%

Service C → CPU 95%

Service D → CPU 15%

Service C may require investigation.

The organization can determine whether the high CPU usage is caused by:

- Inefficient code
- Excessive traffic
- Poor configuration
- Memory problems
- Unnecessary background processing

After optimization, resource consumption can be measured again.

3.4 Sustainable DevOps Feedback Loop

Deploy Applications



Collect Metrics



Measure Resource Usage



Identify Waste



Optimize Application/Infrastructure



Reduce Resources



Measure Again



Continuous Improvement

This creates a continuous sustainability cycle.

3.5 Comparison of the Three Practices

Sustainable DevOps Practice	Main Action	Energy Benefit	Cost Benefit
Autoscaling	Dynamically adjust resources	Avoids unused resources	Reduces unnecessary cloud instances
Efficient Containers	Improve resource utilization	Fewer resources required	Lower infrastructure cost
Monitoring & Optimization	Identify resource waste	Reduces inefficient workloads	Controls and reduces cloud spending

3.6 Additional Sustainable DevOps Techniques

Other techniques can also be introduced:

Right-Sizing

Select cloud instances according to actual workload requirements instead of over-provisioning.

Serverless Computing

For suitable event-driven workloads, serverless services can eliminate the need to continuously operate dedicated servers.

Efficient Database Management

Remove unused databases, optimize queries, and use appropriate storage configurations.

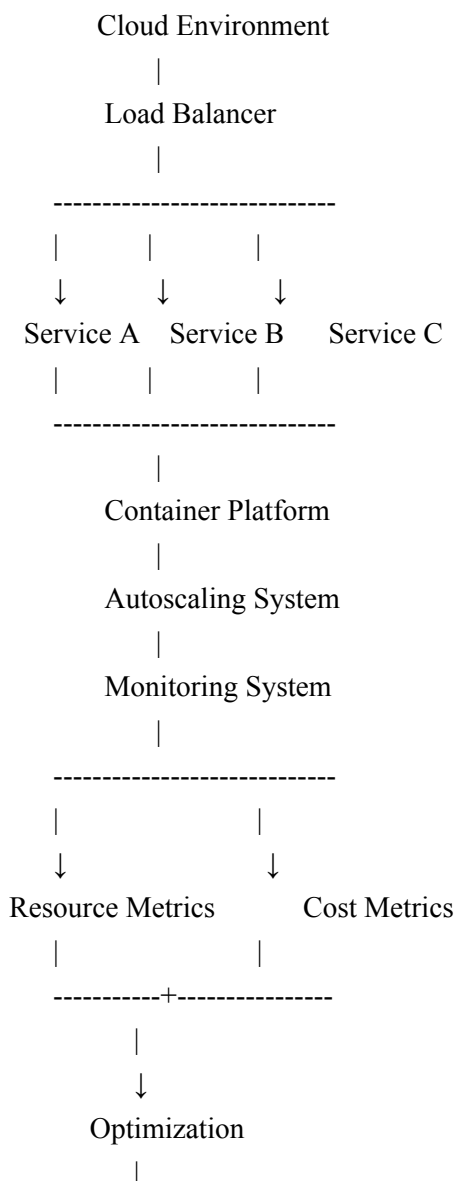
Automated Shutdown

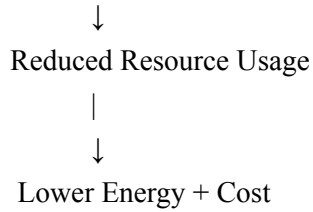
Development and testing environments can be automatically shut down outside working hours.

Efficient CI/CD Pipelines

CI/CD jobs can be optimized so that unnecessary builds and repeated tasks are avoided.

3.7 Overall Sustainable Architecture





Conclusion

Sustainable DevOps aims to maintain software performance and reliability while reducing unnecessary resource consumption and environmental impact.

For the given cloud-based microservices system, the company should implement **autoscaling and resource optimization, efficient containerization and workload scheduling, and continuous monitoring and optimization.**

These practices prevent over-provisioning, improve infrastructure utilization, reduce energy consumption, and lower cloud costs. Sustainable DevOps therefore provides both environmental and economic benefits while maintaining the required application performance.